PBT205 Task 3 Report: RabbitMQ Contact Tracing

Oct 21, 2026 · @Seyla Song

1. Overview

The prototype is a contact tracing system made of three command-line apps that talk only through a RabbitMQ broker. People move around a square board; a tracker logs every time two people stand on the same square, and a query app asks the tracker who a given person has met. It is written in Python (464 lines across four files) and uses the `pika` client library.

App	Lines	Role
`person.py`	111	One person. Starts on a random square and moves one square at a time, publishing each move
`tracker.py`	204	Keeps the board, logs contacts, answers queries
`query.py`	72	Asks the tracker who a person has met, prints the answer and exits
`common.py`	77	Exchange names, argument helpers, connect and subscribe helpers

In scope

- Start-up from the command line, with the middleware endpoint (port, optional `--host`) as the first argument of every app.
- Any number of people moving at their own speed on a board of 1 × 1 up to 1,000 × 1,000.
- Contact detection whenever a person moves onto an occupied square, logged for both people.
- Queries answered newest contact first, each person listed once with the time, square and count of contacts.
- Clear errors for bad arguments, an unreachable broker and badly formed messages.
- Clean exit on Ctrl+C: a person tells the tracker it has left the board.

Extensions built: a `--host` argument (Task 1 was fixed to `localhost`), speed presets (`slow`, `normal`, `fast`) as well as numbers, repeat-contact counts, and request ids so several query apps can run at once without seeing each other's answers.

Out of scope: a graphical board view, saving contacts to disk, and checking that names are unique.

`main.py` is the PyCharm sample script created with the project. It is not part of the Task 3 solution.

2. Requirements and constraints

The brief sets the middleware (RabbitMQ) and the three apps. Each design choice below follows from a requirement, and each one adds a constraint that the next choice has to respect.

Requirement	What it forces	Resulting constraint
Middleware is RabbitMQ, run in Docker	Apps speak AMQP 0-9-1 through `pika`	The broker must be running before any app starts
Apps communicate only through topics	Three exchanges: `position`, `query`, `query-response`	Apps never call each other; every app must agree on the exchange names in `common.py`
The tracker sees every move	`position` is a `fanout` exchange, one queue per subscriber	A later subscriber (a board view) gets every move too, with no change to the tracker
A query gets only its own answer	`query-response` is `direct`, routing key = a random request id	Each query app subscribes before it publishes, or the answer could be lost
Contact = two people on one square at the same time	The tracker keeps a live view of who is on each square	Positions are only as current as the last move received
Board size is configurable, up to 1,000 × 1,000	`--size` checked by `board_size()`; squares stored only when occupied	The tracker and every person must use the same size

Requirement	What it forces	Resulting constraint
A person moves like a chess king	`next_square()` picks only from the neighbouring squares that stay on the board	8 choices in the middle, 5 on an edge, 3 in a corner, 0 on a 1 × 1 board
Speed is configurable	Moves per second, or `slow` / `normal` / `fast`	Speed must be greater than 0

Protocol. AMQP 0-9-1 over TCP on port 5672 (RabbitMQ, n.d.-a). Port 15672 serves the management UI, which was used to inspect exchanges and queues during testing.

Serialisation (JSON). All messages are UTF-8 JSON, as in Task 1 (Bray, 2017). It ships with Python's `json` module (Python Software Foundation, n.d.-b), is easy to read in the management UI, and lets messages gain new fields without breaking older apps. Board coordinates are integers and times are Unix timestamps (seconds as a float), so no date parsing is needed between apps.

Integration. The broker is the only shared component. Any program that publishes the same JSON to `position` takes part as a person, and any program that subscribes to `position` (a monitor or a board view) can be added without changing the others.

Boundary conditions

- Port not a number, board size outside 1–1000, or speed not a number above 0: `argparse` prints usage and an error, exit code 2.
- Broker not reachable at start-up: print the Docker command to start it and exit with code 1.
- Broker lost during a session: print `Lost connection to RabbitMQ.` and exit.
- Move off the board, bad JSON or a missing field reaching the tracker: printed and ignored; the tracker keeps running.
- A person staying on the same square: not a new contact.
- No tracker running when a query is made: the query app waits `--timeout` seconds (default 5) then exits with code 1.
- A person who leaves: removed from the board, contact history kept.

3. Design

Three exchanges carry all traffic. Every person publishes moves to the `position` fanout exchange; the tracker reads them through its own queue. A query app publishes a name to the `query` fanout exchange, and the tracker answers on the `query-response` direct exchange using the query's request id as the routing key, so only the app that asked receives the answer.

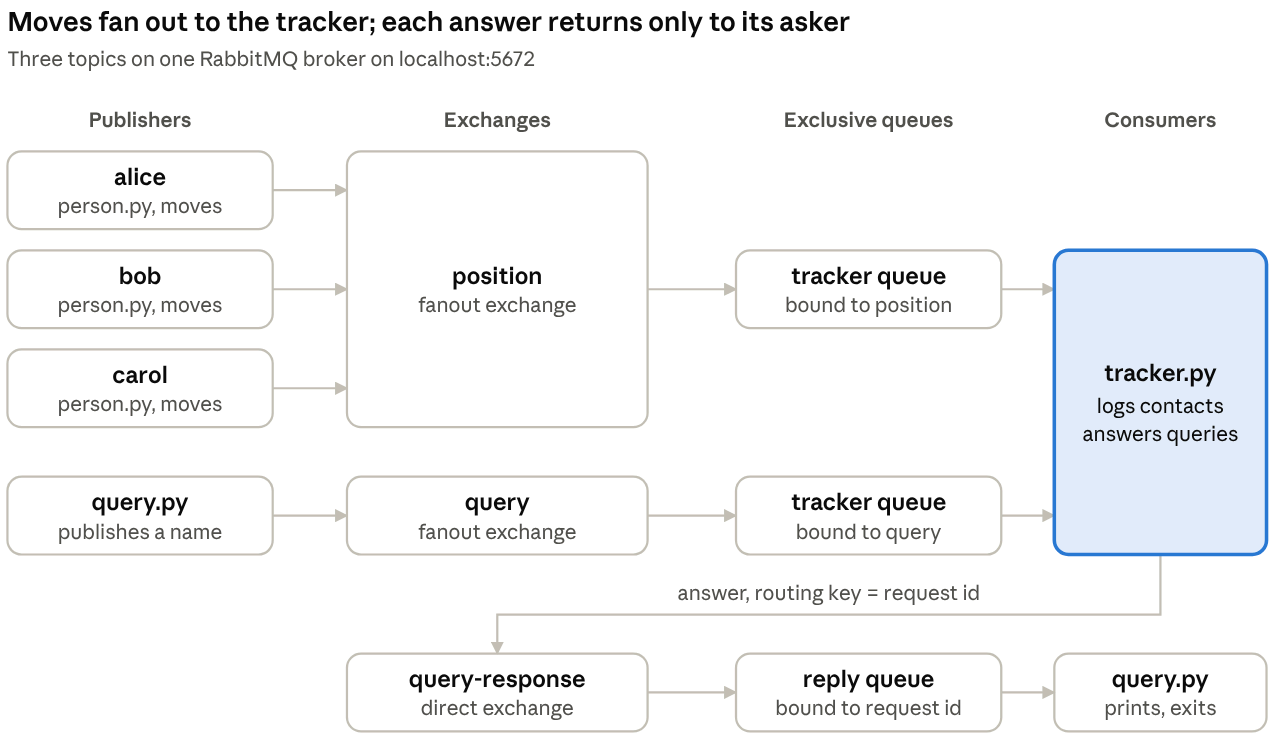


Figure 1. Architecture: three exchanges, one exclusive queue per subscriber

Every move reaches the tracker through its own queue. The tracker's answer goes to `query-response` with the request id as the key, so it lands only in the reply queue of the query app that asked.

Why these exchange types

Topic	Option	What happens	Verdict
`position`	One shared queue	Fine for one tracker, but a second subscriber (a board view) would steal half the moves	Rejected

Topic	Option	What happens	Verdict
`position`	`direct` with the person's name as key	Subscribers must bind each name, or know them in advance	Rejected: the tracker cannot know who will join
`position`	`fanout`, a queue per subscriber	Every subscriber gets every move; new subscribers need no change to anyone	**Chosen**
`query`	`fanout`	Same pattern as `position`; any number of trackers or loggers can listen	**Chosen**
`query-response`	`fanout`	Every query app would see every answer and have to filter	Rejected
`query-response`	`direct`, routing key = request id	Each query app binds only its own id; concurrent queries never mix	**Chosen**

Every subscriber declares its queue with `queue=""` and `exclusive=True` (`common.py:67-77`), as in Task 1, so the broker deletes it when the app exits.

Contact detection

The tracker's state lives in the `ContactTracker` class (`tracker.py:32-114`), kept apart from RabbitMQ so it can be unit tested. It holds three dictionaries:

Structure	Maps	Used for
`positions`	person → `(x, y)`	Where everyone is now
`squares`	`(x, y)` → set of people	Who is on each occupied square
`contacts`	person → list of contacts, oldest first	The history a query reads

When a person moves, the tracker removes them from their old square, looks up the new square in `squares`, and logs a contact for both sides with everyone already there. Only one square is checked, so a move costs the same on a 10 × 10 board as on a 1,000 × 1,000

board. Only occupied squares are stored, so a large board uses no extra memory. Moving to the square you are already on is not a new contact.

A query walks the person's contact list once, keeps the latest contact and a count per name, then sorts newest first.

Message contracts

Every body is a UTF-8 JSON object. Times are Unix timestamps in seconds.

```
// position: a move
{ "type": "move", "person": "alice", "x": 3, "y": 4, "timestamp":
1792573000.1 }
// position: a person stopped
{ "type": "leave", "person": "alice", "timestamp": 1792573010.0 }
// query
{ "person": "alice", "request_id": "6f1c…" }
// query-response
{ "request_id": "6f1c…", "person": "alice", "known": true,
  "contacts": [ { "name": "bob", "x": 3, "y": 4, "timestamp": 1792573005.2,
"count": 2 } ] }
```

- `type`: `move` or `leave`. A `leave` removes the person from the board but keeps their history.
- `known`: false when the tracker has never seen that name, so the query app can tell "unknown" apart from "met nobody".
- The tracker logs contacts with its own clock at the time it receives the move, not the person's `timestamp`, so clocks on different machines cannot disagree.

Terminal interface

There is no GUI. The tracker prints `CONTACT at (1, 2): alice met bob` for each contact and prints every query it answers. Each person prints its moves. The query app prints a list such as `bob last at (2, 1) at 20:14:56, 2 times` and exits.

4. Implementation

The solution is four Python files written with `pika` 1.4.4. It was tested on Python 3.14.8 against RabbitMQ 3.13.7 (Docker image `rabbitmq:3-management`) on macOS 26.5.

Start-up arguments

App	Argument	Required	Meaning	Check
all	`port`	Yes	RabbitMQ TCP port, normally 5672	Must parse as an integer
all	`--host`	No	RabbitMQ host, default `localhost`	None
tracker, person	`--size`	No	Board width and height, default 10	Whole number from 1 to 1000
person	`name`	Yes	Unique person identifier	None (any string)
person	`speed`	No	Moves per second, or `slow` (0.5), `normal` (1), `fast` (2)	A preset or a number above 0
query	`name`	Yes	Person to look up	None
query	`--timeout`	No	Seconds to wait for the tracker, default 5	Must parse as a number

How to run it

1. Start the broker: `docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-management`
2. Create a virtual environment and install `pika`: `python3 -m venv venv`, `source venv/bin/activate`, then `pip install -r requirements.txt`.
3. Start the tracker: `python tracker.py 5672 --size 5`
4. Start one person per terminal: `python person.py 5672 alice fast --size 5`
5. Ask about anyone at any time: `python query.py 5672 alice`

Key code

Logging a contact for both people on the new square (`tracker.py:74-80`):

```
people_here = self.squares.setdefault(new_square, set())
met = sorted(people_here)
for other in met:
    # A contact goes both ways.
    self._log(person, other, x, y, timestamp)
    self._log(other, person, x, y, timestamp)
people_here.add(person)
```

Picking only king moves that stay on the board (`person.py:54-61`):

```
moves = [(x + dx, y + dy) for dx, dy in DIRECTIONS
         if 0 <= x + dx < size and 0 <= y + dy < size]
if not moves:
    return x, y
return random.choice(moves)
```

Subscribing to the answer before asking, so it cannot be missed (`query.py:43-49`):

```
request_id = str(uuid.uuid4())
response_queue = subscribe(channel, QUERY_RESPONSE_EXCHANGE,
routing_key=request_id)
channel.basic_publish(exchange=QUERY_EXCHANGE, routing_key="",
                      body=json.dumps({"person": args.name, "request_id":
request_id}))
```

Waiting between moves without dropping the connection (`person.py:94-96`):

```
# connection.sleep() keeps the connection alive while waiting,
# unlike time.sleep().
connection.sleep(interval)
```

A single thread is enough in every app. The tracker consumes two queues on one channel with `start_consuming()`, the person publishes in a loop, and the query app uses `consume()` with `inactivity_timeout`. This avoids the cross-thread hand-off Task 1 needed.

Attribution

- The connect, declare, publish and consume pattern follows the official RabbitMQ Python tutorials 3 and 4 (RabbitMQ, n.d.-b, n.d.-c). The request id used as a routing key is adapted from the RPC tutorial's correlation id (RabbitMQ, n.d.-d).

- `connection.sleep()` and `consume(inactivity_timeout=…)` are used as described in the `pika` documentation (Pika, n.d.).
- Argument checking uses Python's `argparse` (Python Software Foundation, n.d.-a).
- The broker runs from the official Docker image (Docker Hub, n.d.).

5. Testing and validation

All 13 unit tests and 15 of 17 integration tests passed against a live broker on 21 October 2026. The two failures are real bugs, listed under "Bugs found" below. All core requirements passed: contacts logged both ways, newest-first queries, concurrent queries, board edges, and a lost broker.

Method

- **Unit tests** (`test_tracker.py` , `python -m unittest -v`): test `ContactTracker` and `next_square()` directly, with fixed timestamps and no broker.
- **Integration tests:** a Python harness started real `tracker.py` , `person.py` and `query.py` processes, published hand-made messages with `pika` , and checked each app's output and exit code. The broker was RabbitMQ 3.13.7 in Docker on `localhost:5672` . Small boards (1 × 1 to 3 × 3) made contacts happen within seconds.
- **Broker checks:** the management HTTP API (`/api/queues`) counted queues before, during and after the apps ran.
- **Broker failure:** a second, throwaway broker was started on port 5673 and stopped with `docker stop` while a tracker and a person were connected. This left the main broker untouched.

Unit test results

Area	Tests	What they check	Result
Contact logic	8	No contact on different squares; logged for both people; three on one square; newest first; repeats listed once with count and latest time; staying put is not a contact; no contact after moving away; leave keeps history	8 pass

Area	Tests	What they check	Result
Board edges	2	Moves to x or y of −1 or 10 on a 10 × 10 board rejected; (999, 999) works and (1000, 999) is rejected on a 1,000 × 1,000 board	2 pass
Movement	3	Always one king move away; never leaves the board from any corner (sizes 2, 10, 1000); stays put on a 1 × 1 board	3 pass

Integration test results

ID	Test	Input / steps	Expected	Actual	Result
T1	Contacts detected	Tracker and alice, bob, carol (`fast`) on a 2 × 2 board for 6 s	`CONTACT` lines in the tracker	18 contacts, e.g. `CONTACT at (0, 1): bob met alice`	Pass
T2	Query order and counts	`python query.py 5672 alice`	Each person once, newest first, with count	`bob … 21:43:19, 8 times`, then `carol … 21:43:18, 5 times`	Pass
T3	Unknown person	Query `zed`	Clear "not seen" message	`No one called 'zed' has been seen by the tracker.`	Pass
T4	Concurrent queries	5 query apps at once (alice, bob, carol, zed, alice)	Each gets its own answer	All 5 correct, exit 0	Pass
T5	1 × 1 board	alice and bob, both stuck on (0, 0) for 3 s	One contact, not one per move	`bob … 1 time`	Pass
T6	Person leaves	Ctrl+C on each person	Tracker removes them, history kept	`alice left the board`; bob's query still lists alice and carol	Pass
T7	Malformed position	Non-JSON, x = 5 on a 2 × 2 board, no `person`, x = `"abc"`, a JSON list	Each ignored, tracker keeps running	5 `Ignored position message …` lines; tracker still running	Pass
T8	Port not a number	`python tracker.py abc`	Error, exit 2	`invalid int value: 'abc'`, exit 2	Pass
T9	Board size out of range	`--size 0`, `--size 1001`	Error, exit 2	`board size must be between 1 and 1000`, exit 2	Pass

ID	Test	Input / steps	Expected	Actual	Result
T10	Bad speed	Speed `0`, speed `zoom`	Error, exit 2	`speed must be greater than 0`; `speed must be slow, normal, fast or a number`	Pass
T11	Missing arguments	`python query.py`	Usage, exit 2	`the following arguments are required: port, name`	Pass
T12	Broker unreachable	Port 5999 (nothing listening)	Clear error, exit 1	Error plus the Docker start command, exit 1	Pass
T13	No tracker running	Query with `--timeout 2`, no tracker	Error after 2 s, exit 1	`no answer from the tracker within 2 s`, after 2.2 s, exit 1	Pass
T14	Broker down mid-session	`docker stop` on the test broker	Both apps tell the user and exit	`Lost connection to RabbitMQ.` from tracker and person, 1.4 s later	Pass
T15	Queues removed on exit	Count queues before, during, after the tracker	0 → 2 → 0	0 → 2 → 0	Pass
T16	Two people with the same name	Two `alice` processes on a 3 × 3 board for 5 s	Treated as two people	Merged into one; query: `alice has not come into contact with anyone.`	**Fail (B2)**
T17	Malformed query	`{"person": [1], …}`, then `{"request_id": 5, …}` published to `query`	Ignored, tracker keeps running	Tracker crashed: `TypeError: cannot use 'list' as a dict key`, then `AssertionError: A non-string value was supplied for self.routing_key`	**Fail (B1)**

Sample output from T1 and T2:

```
# tracker (2 x 2 board)
CONTACT at (0, 1): bob met alice
CONTACT at (1, 1): carol met alice
CONTACT at (0, 0): alice met bob

# query
alice has come into contact with (most recent first):
  bob          last at (0, 0) at 21:43:19, 8 times
  carol        last at (0, 1) at 21:43:18, 5 times
```

Bugs found

ID	Bug	Cause	Impact	Fix
B1	One bad query crashes the tracker	`on_query` (`tracker.py:164-184`) checks that `person` and `request_id` exist but not their types. A list `person` fails at `tracker.py:173`; a number `request_id` fails in `basic_publish` at `tracker.py:183`. Both errors are outside the `try`, and `main()` only catches `AMQPError`	Any program on the broker can stop contact tracing with one message, and all contacts in memory are lost	Check `isinstance(person, str)` and `isinstance(request_id, str)` inside the `try` and ignore the message otherwise
B2	Two people with the same name become one	The tracker keys everything on the name	Their contacts with each other are never logged, and when one leaves the other vanishes from the board until its next move	Add a per-process id (`uuid4()`) to each move, and have the tracker warn about or reject a second process using a name already in use

How the results change the solution

- **B1** shows that checking a field exists is not enough; its type must be checked too. `on_position` already does this by passing coordinates through `int()` inside its `try` (T7), and `on_query` needs the same care.

- **B2** repeats Task 1's lesson: a display name is not an identity. The README lists it as a limitation, but the tracker could detect it.
- T7 confirmed that validating every position message works: five kinds of bad message were ignored without a crash.
- T4 confirmed the choice of a `direct` exchange keyed by request id: five concurrent queries never saw each other's answers.
- T15 confirmed the choice of `exclusive=True` queues: nothing is left on the broker after the apps exit.

6. Hardware and software constraints

The prototype depends on one broker and one tracker, and keeps everything in memory. That is fine for a classroom demo, but it limits availability, history and scale.

Constraint	Detail	Effect	Mitigation
Docker dependency	The broker runs in the `rabbitmq:3-management` container	No Docker (or a port clash on 5672) means no tracing	A native RabbitMQ install, or a hosted broker
Single broker, single tracker	Every app connects to one broker; one tracker holds all state	If either stops, tracing stops (T14)	A RabbitMQ cluster with quorum queues, and a standby tracker
No persistence	Contacts live in the tracker's memory; exchanges are not durable, queues are exclusive and `auto_ack=True`	Restarting the tracker clears all contacts; moves published while it is down are lost	Write contacts to a database, and use a durable queue with manual acks
Board size must match	The tracker and each person take `--size` separately	A person on a bigger board has its outer moves ignored (T7)	Have the tracker publish its size for persons to read at start-up
Names are identities	No check that a name is unique	Two people with one name merge (B2)	A per-process id

Constraint	Detail	Effect	Mitigation
No authentication	Any program that can reach the broker can publish moves or queries	False contacts, and the crash in B1	Per-app broker accounts and permissions
Localhost by default	`--host` exists, but RabbitMQ's `guest` user is limited to localhost	Apps on other machines need a broker user	A dedicated broker user per app
Scaling	Each move is one message; the tracker checks one square per move	Work grows with people × speed, not board size. Contact history grows without limit	Fine for tens of people; larger loads need load testing (not done) and a cap or expiry on history
Python and pika	Needs Python 3.8+ and `pika` 1.4.x	Each machine needs Python installed	Package the apps, for example with PyInstaller

The hardware used for testing was a single Mac laptop running the broker and all apps. No performance limits were reached at the volumes tested (up to 3 people at 2 moves per second and 5 concurrent queries).

7. Conclusion and lessons learned

The prototype meets the Task 3 brief. People move around the board at their own speed, the tracker logs every contact both ways, and queries return each contact newest first, all through RabbitMQ topics. The apps handle bad arguments, bad position messages and a lost broker cleanly. Testing found two bugs, each with a small fix.

- The exchange type decides who hears what. `fanout` for moves lets any number of subscribers watch the board; `direct` keyed by request id gives each query a private reply channel, with no filtering needed.
- Keeping the contact logic in its own class made it easy to unit test without a broker, and the `squares` index keeps each move's cost flat even on a 1,000 × 1,000 board.
- A message-driven design can avoid threads. One channel consuming two queues was enough, unlike Task 1's input thread.

- Validate every field's type, not just its presence. `on_position` did this and survived five kinds of bad message (T7); `on_query` did not and crashed (B1).
- Testing real processes against a real broker found both bugs; the unit tests, which all passed, could not.

8. References and appendix

References

Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) data interchange format* (RFC 8259). Internet Engineering Task Force. https://doi.org/10.17487/RFC8259

Docker Hub. (n.d.). *rabbitmq: Official image*. Retrieved October 21, 2026, from https://hub.docker.com/_/rabbitmq

Pika. (n.d.). *BlockingConnection*. Pika documentation. Retrieved October 21, 2026, from https://pika.readthedocs.io/en/stable/modules/adapters/blocking.html

Python Software Foundation. (n.d.-a). *argparse: Parser for command-line options, arguments and subcommands*. Python documentation. Retrieved October 21, 2026, from https://docs.python.org/3/library/argparse.html

Python Software Foundation. (n.d.-b). *json: JSON encoder and decoder*. Python documentation. Retrieved October 21, 2026, from https://docs.python.org/3/library/json.html

RabbitMQ. (n.d.-a). *AMQP 0-9-1 model explained*. Retrieved October 21, 2026, from https://www.rabbitmq.com/tutorials/amqp-concepts

RabbitMQ. (n.d.-b). *RabbitMQ tutorial: Publish/subscribe (Python)*. Retrieved October 21, 2026, from https://www.rabbitmq.com/tutorials/tutorial-three-python

RabbitMQ. (n.d.-c). *RabbitMQ tutorial: Routing (Python)*. Retrieved October 21, 2026, from https://www.rabbitmq.com/tutorials/tutorial-four-python

RabbitMQ. (n.d.-d). *RabbitMQ tutorial: Remote procedure call (RPC) (Python)*. Retrieved October 21, 2026, from https://www.rabbitmq.com/tutorials/tutorial-six-python

Appendix A: Test environment

Item	Version
OS	macOS 26.5.2
Python	3.14.8
pika	1.4.4
RabbitMQ	3.13.7, Docker image `rabbitmq:3-management`